

Department of Computer Science

Periyar Nagar, Vallam Thanjavur-613403,Tamilnadu, India

Email: headmcs@pmu.edu web: www.pmu.edu

Phone: +91-4362-264600,Fax: +91-4362-264660

**PERIYAR
MANIAMMAI**INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University)

think • innovate • transform

DEPARTMENT OF COMPUTER SCIENCE

Reg No : 124011173108

Name : M.sownthariya

Year / Sem : III/V

Course : B.Sc. AI

Course Code : U24AI303

Course Name : Neural Networks

Topic : Types of Activation Functions and Terminology of Neural Networks.

Date : 07/09/2026

1. INTRODUCTION

Artificial Intelligence (AI) is an important area of computer science concerned with building systems that can perform tasks associated with human intelligence. Machine Learning (ML), a major branch of AI, enables computers to learn patterns from data and use those patterns to make predictions or decisions.

Artificial Neural Networks (ANNs) are machine-learning models inspired by the way biological neurons process information. An ANN contains interconnected artificial neurons arranged in layers. By adjusting numerical parameters during training, a neural network can learn useful representations from examples.

Neural networks are widely used in image recognition, speech recognition, natural language processing, medical analysis, recommendation systems, fraud detection, forecasting, and many other applications. Their ability to model complex relationships makes them particularly useful for high-dimensional data.

A typical neural network contains an input layer, one or more hidden layers, and an output layer. Each neuron combines input values using weights and a bias and then applies an activation function.

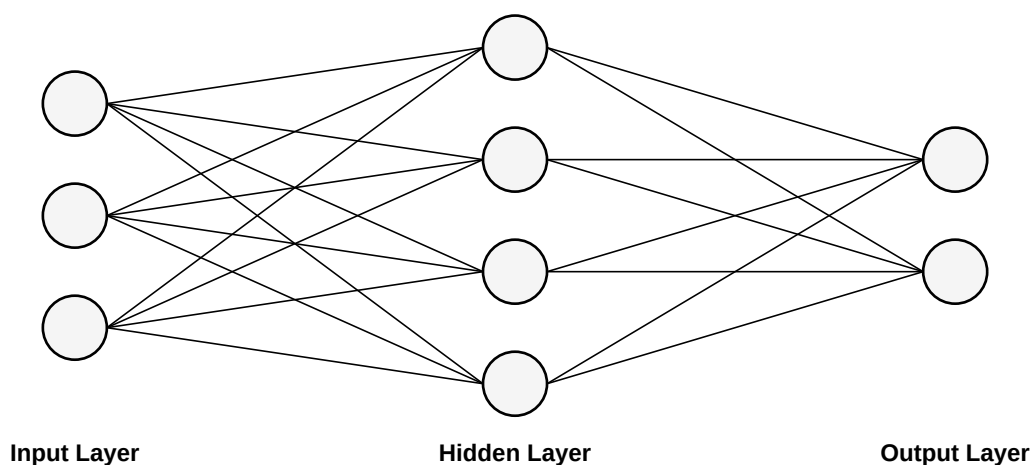


Figure 1: Basic architecture of an artificial neural network.

Activation functions are especially important because they introduce non-linearity. Without a non-linear activation function, stacking multiple linear layers would still produce a linear transformation, limiting what the network can learn.

This assignment explains the major types of activation functions and the terminology needed to understand how neural networks are constructed and trained.

2. ACTIVATION FUNCTIONS

An activation function is a mathematical function applied to the weighted input of a neuron. It determines the neuron's output and, more importantly, allows a neural network to represent non-linear relationships.

For a neuron, the weighted sum is commonly written as:

$$z = \sum(w \cdot x) + b$$

where x represents an input, w represents its weight, and b represents the bias. The activation function then produces the neuron output:

$$a = f(z)$$

The choice of activation function affects gradient flow, training speed, numerical behavior, and sometimes the final performance of a model. There is no single activation function that is best for every layer and every task.

2.1 IMPORTANCE OF ACTIVATION FUNCTIONS

- They introduce non-linearity so the network can learn complex relationships.
- They transform a neuron's weighted sum into an output suitable for the next layer.
- They influence how gradients flow during backpropagation.
- They can affect training speed and numerical stability.
- They are selected according to the architecture and learning task.

2.2 COMMON ACTIVATION FUNCTIONS

The commonly discussed activation functions include Sigmoid, Tanh, ReLU, Leaky ReLU, ELU, and Softmax. Their mathematical shapes and output ranges are different, so each has particular strengths and limitations.

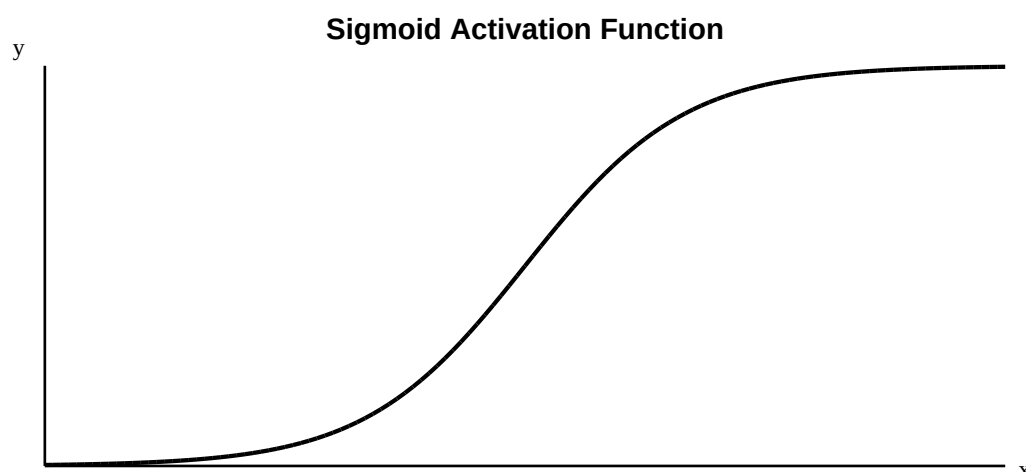


Figure 2: Sigmoid activation function.

3. SIGMOID AND TANH

3.1 SIGMOID FUNCTION

The Sigmoid function maps any real-valued input into a value between 0 and 1. Its formula is:

$$f(x) = 1 / (1 + e^{-x})$$

Because its output is bounded between 0 and 1, sigmoid is useful when a model needs a probability-like output, especially for binary classification.

- Output range: 0 to 1.
- Useful for many binary classification output layers.
- Simple and easy to interpret.
- Can suffer from vanishing gradients when inputs are very large or very small.
- It is not zero-centered, which can make optimization less convenient in some hidden-layer settings.

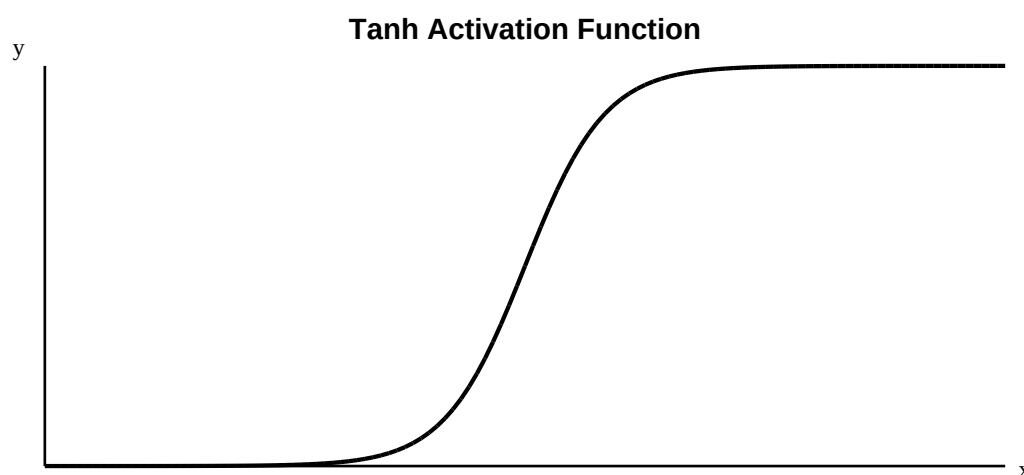


Figure 3: Tanh activation function.

3.2 TANH FUNCTION

Tanh, or hyperbolic tangent, maps inputs into the range -1 to +1. It is zero-centered and historically has been used in hidden layers.

$$f(x) = \tanh(x)$$

- Output range: -1 to +1.
- Zero-centered output.
- Can be preferable to sigmoid in some hidden-layer situations.
- Still susceptible to vanishing gradients for saturated inputs.

4. RELU AND LEAKY RELU

4.1 RELU FUNCTION

ReLU stands for Rectified Linear Unit. It is widely used in hidden layers of modern deep neural networks because it is simple and computationally efficient.

$$f(x) = \max(0, x)$$

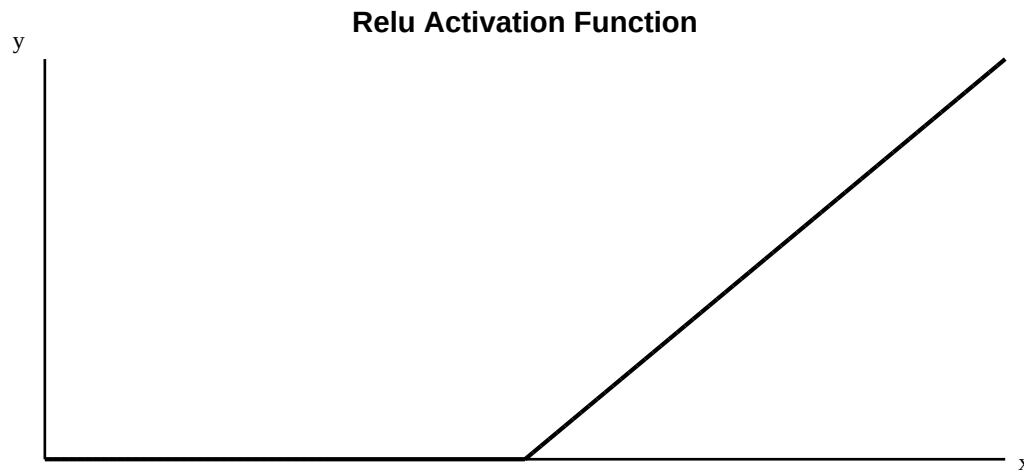


Figure 4: ReLU activation function.

For positive inputs, ReLU returns the input itself. For negative inputs, it returns zero. This simple behavior often provides efficient gradient-based learning.

- Computationally simple.
- Widely used in hidden layers.
- Helps reduce the saturation problem seen in sigmoid and tanh for positive inputs.
- Can produce inactive neurons when they repeatedly receive negative inputs.

4.2 LEAKY RELU

Leaky ReLU modifies ReLU by allowing a small negative output for negative inputs. A common form is:

$$f(x) = x, \text{ if } x > 0; \alpha x, \text{ if } x \leq 0$$

Here α is a small positive constant. This small negative slope helps maintain a gradient on the negative side and can reduce the dying-ReLU problem.

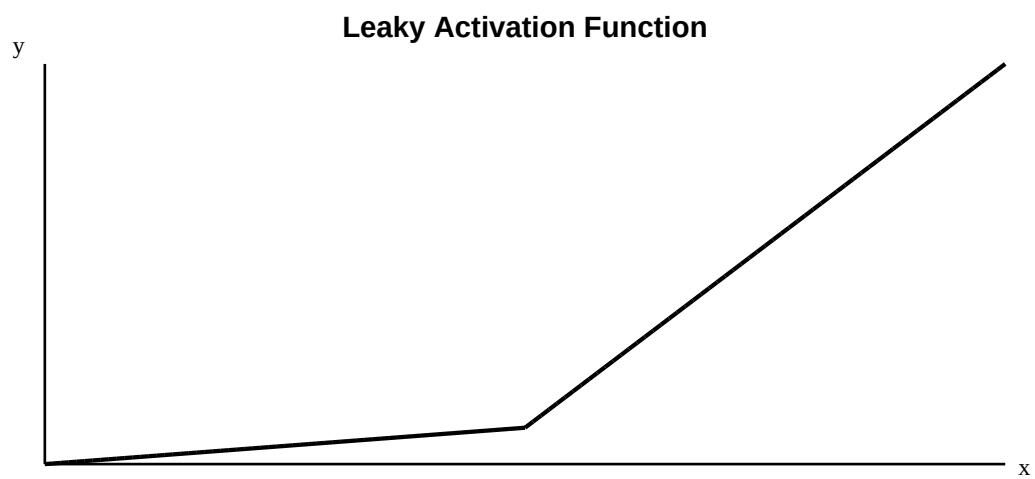


Figure 5: Leaky ReLU activation function.

5. ELU AND SOFTMAX

5.1 ELU FUNCTION

ELU stands for Exponential Linear Unit. It behaves linearly for positive inputs and uses an exponential expression for negative inputs.

$$f(x) = x, \text{ if } x > 0; \alpha(e^x - 1), \text{ if } x \leq 0$$

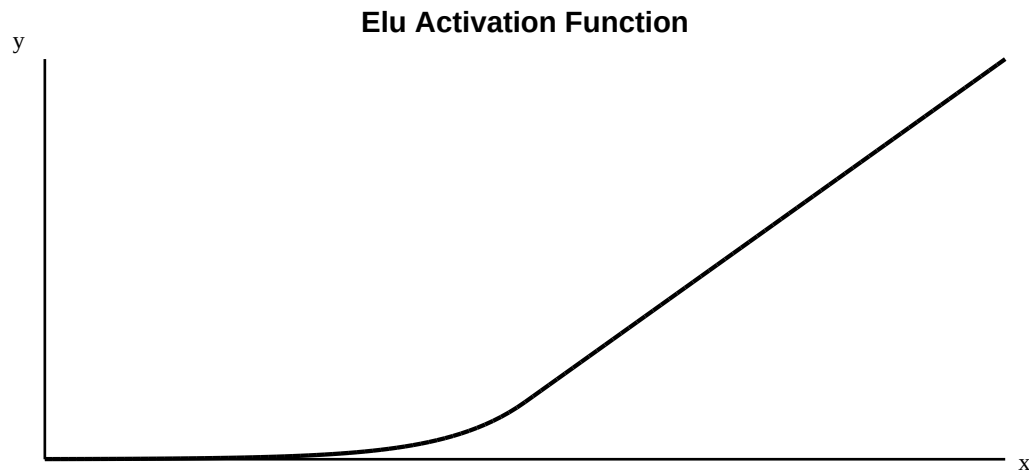


Figure 6: ELU activation function.

- Allows negative outputs for negative inputs.
- Has a smooth negative region.
- Can reduce some limitations associated with standard ReLU.
- More computationally expensive than ReLU because of the exponential operation.

5.2 SOFTMAX FUNCTION

Softmax is commonly used in the output layer for multi-class classification. It converts a set of scores into values that form a probability distribution.

$$\text{Softmax}(z_j) = e^{z_j} / \sum_k e^{z_k}$$

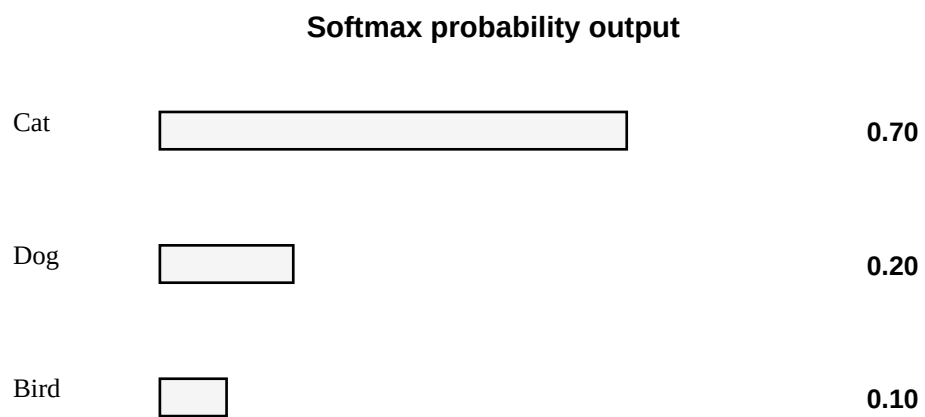


Figure 7: Example of Softmax probabilities for three classes.

For example, a classifier might produce probabilities of 0.70 for Cat, 0.20 for Dog, and 0.10 for Bird. The values sum to 1, and the class with the highest probability is selected as the prediction.

6. COMPARISON OF ACTIVATION FUNCTIONS

Different activation functions have different output ranges, properties, and typical uses. The following table provides a concise comparison.

Function	Output Range	Typical Use	Key Point
Sigmoid	0 to 1	Binary classification output	Probability-like output
Tanh	-1 to +1	Some hidden layers	Zero-centered
ReLU	0 to ∞	Hidden layers	Simple and efficient
Leaky ReLU	Small negative to ∞	Hidden layers	Keeps small negative slope
ELU	Approximately $-\alpha$ to ∞	Hidden layers	Smooth negative region
Softmax	0 to 1	Multi-class output	Probabilities sum to 1

7. HOW TO CHOOSE AN ACTIVATION FUNCTION

Activation functions should be chosen based on the role of the layer and the task being solved. In many deep networks, ReLU or one of its variants is a practical default for hidden layers. Sigmoid is commonly used for binary classification outputs, while Softmax is commonly used when exactly one class must be selected from multiple classes.

The correct choice is not determined by a single rule. Architecture, data, optimization method, and task all matter. Modern libraries also provide additional activation functions, and model performance should ultimately be evaluated experimentally.

A useful principle is to avoid treating activation functions as interchangeable. Their gradient behavior and output ranges can materially affect training.

8. TERMINOLOGY OF NEURAL NETWORKS

8.1 NEURON

A neuron is the basic processing unit of an artificial neural network. It receives inputs, multiplies them by weights, adds a bias, and applies an activation function.

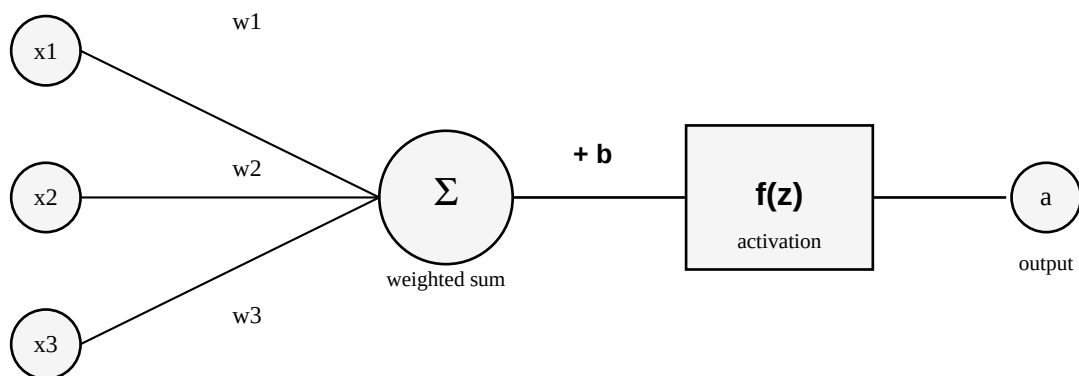


Figure 8: Structure of an artificial neuron.

8.2 INPUT

Input refers to the information supplied to the network. In an image classification task, inputs may be pixel values or extracted features. In a house-price model, inputs might include area, number of rooms, and location-related features.

8.3 WEIGHTS

Weights determine how strongly individual inputs influence a neuron. During training, the network adjusts weights to reduce prediction error.

8.4 BIAS

Bias is an additional learnable parameter added to the weighted sum. It gives the neuron flexibility to shift its activation.

8.5 LAYER

A layer is a collection of neurons operating at the same stage. The input layer receives data, hidden layers transform representations, and the output layer produces the final prediction.

8.6 HIDDEN LAYER

A hidden layer is any layer between the input and output layers. Multiple hidden layers allow a network to build increasingly complex representations of the input.

9. TRAINING TERMINOLOGY

9.1 EPOCH

An epoch is one complete pass through the entire training dataset. Training for multiple epochs means the model sees the training data repeatedly.

9.2 BATCH SIZE

Batch size is the number of training examples processed before the model performs a parameter update. Common values include 16, 32, 64, and 128, although the best choice depends on the problem and available hardware.

9.3 LEARNING RATE

Learning rate controls the size of parameter updates. A rate that is too large can make optimization unstable, while a rate that is too small can make training unnecessarily slow.

9.4 LOSS FUNCTION

A loss function measures the difference between predicted and target values. Training attempts to minimize this loss. Examples include Mean Squared Error, Binary Cross-Entropy, and Categorical Cross-Entropy.

9.5 OPTIMIZER

An optimizer updates weights and biases using gradient information. Common optimizers include Gradient Descent, Stochastic Gradient Descent, RMSProp, and Adam.

9.6 FORWARD PROPAGATION

Forward propagation is the process of passing input data through the network from the input layer to the output layer to obtain a prediction.

9.7 BACKPROPAGATION

Backpropagation computes gradients of the loss with respect to model parameters and propagates this information backward through the network. The optimizer then uses those gradients to update parameters.



Figure 9: Simplified neural network training workflow.

10. GENERALIZATION AND MODEL PERFORMANCE

10.1 TRAINING DATA

Training data is the portion of a dataset used to learn model parameters. The network adjusts weights and biases based on the examples in this set.

10.2 VALIDATION DATA

Validation data is used during model development to compare configurations and monitor generalization. It should be kept separate from the training examples used to fit the parameters.

10.3 TEST DATA

Test data is used for final evaluation on examples that were not used to fit the model. A separate test set provides a more honest estimate of performance on unseen data.

10.4 OVERFITTING

Overfitting occurs when a model learns the training data too closely and performs worse on unseen data. It can result from excessive model complexity, limited data, or overly long training.

Common ways to reduce overfitting include collecting more data, using regularization, applying dropout where appropriate, using early stopping, and reducing unnecessary model complexity.

10.5 UNDERFITTING

Underfitting occurs when a model is too simple or insufficiently trained to capture important patterns. It usually results in poor performance on both training and unseen data.

10.6 DEEP LEARNING

Deep Learning uses neural networks with multiple layers. Different layers can learn progressively more abstract representations. For example, in image tasks, earlier layers may learn edges, intermediate layers may learn shapes, and later layers may represent higher-level objects.

11. APPLICATIONS OF NEURAL NETWORKS

- Image recognition and computer vision.
- Natural language processing and text classification.
- Speech recognition and voice interfaces.
- Medical image analysis and decision support.
- Recommendation systems.
- Fraud and anomaly detection.
- Autonomous and intelligent systems.

12. CONCLUSION

Artificial Neural Networks are a fundamental technology in Artificial Intelligence and Deep Learning. Their ability to learn from examples comes from interconnected neurons whose parameters are adjusted during training.

Activation functions are essential because they introduce non-linearity and control how neuron outputs are transformed. Sigmoid, Tanh, ReLU, Leaky ReLU, ELU, and Softmax each have different characteristics. ReLU and related functions are widely used in hidden layers, while sigmoid and Softmax are commonly associated with classification outputs.

Understanding neural network terminology is equally important. Concepts such as weights, bias, layers, epochs, batch size, learning rate, loss functions, optimizers, forward propagation, backpropagation, overfitting, and underfitting describe the main components and processes involved in building and training neural networks.

Neural networks are now applied to computer vision, natural language processing, speech recognition, healthcare, recommendation systems, fraud detection, and autonomous systems. A strong understanding of activation functions and neural network terminology therefore provides a useful foundation for further study and practical AI development.

13. REFERENCES

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press.
2. Michael Nielsen, *Neural Networks and Deep Learning*.
3. Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*.
4. Stanford University, *CS231n: Convolutional Neural Networks for Visual Recognition*.
5. Google Developers, *Machine Learning Crash Course*.